

Bitácora de decisiones — Sandbox SPEI (PRAXTHON AMATEUR 2026)

¿Qué se decidió?

- **Arquitectura en capas** (controller → service → validation/engine → repository), con separación estricta entre DTO y entidad.
- **Procesamiento asíncrono disparado después del commit** (TransactionSynchronizationManager): el cliente recibe 201 de inmediato en estado RECIBIDO; el avance a EN_PROCESO y al estado final ocurre en segundo plano.
- **Motor de escenarios (EscenarioResolver) y máquina de estados (MáquinaEstados) desacoplados** de la persistencia: son funciones deterministas, fáciles de razonar y de probar aisladas.
- **Bloqueo pesimista (SELECT ... FOR UPDATE)** tanto en el avance automático como en la transición manual (POST /operaciones/{id}/transiciones), con verificación del estado antes de actuar.
- **Validación acumulativa**: se reportan todos los errores de un request en un solo 422, en vez de fallar en el primer campo inválido.
- **Idempotencia como capa aparte** (IdempotenciaService), evaluada antes de validar.

¿Por qué?

- Separar DTO/entidad evita que un campo interno (p. ej. escenarioResuelto) se filtre sin querer.
- El post-commit async evita que una operación "fantasma" dispare procesamiento si la transacción se revierte, y evita que el cliente espere los segundos de simulación para recibir respuesta.
- El bloqueo pesimista + verificación de estado existe porque detectamos que el avance automático y una transición manual podían competir por la misma operación; sin esto, una pudiera pisar a la otra.
- La validación acumulativa da mejor experiencia de depuración a quien consume la API: corrige todo de una vez, no campo por campo.

Qué se dejó fuera (y bajo qué criterio)

- **Catálogo cerrado de tipos de documento de identidad** (INE, PASAPORTE, etc.): hoy solo se valida que tipo sea alfabético, no que pertenezca a una lista cerrada.
- **Validación de la fecha embebida en el RFC** (posiciones 4-9): se valida el formato con regex, no que la fecha sea calendáricamente válida.

Intentos que no funcionaron

- Los campos nombre, concepto y documentoIdentidad inicialmente solo validaban longitud, no contenido: en pruebas manuales se coló texto con emojis y símbolos (#\$/). Se corrigió agregando validación por expresión regular (`\p{L}` para letras Unicode) en vez de depender de los límites de longitud como única defensa.
- `identificacionFiscal` se intentó mandar como campo suelto antes de existir en el modelo: Jackson lo descartaba en silencio (por el default de Spring Boot de ignorar propiedades desconocidas) sin avisar error alguno. Se corrigió agregándolo formalmente al DTO, la entidad y el validador, en vez de asumir que "si no truena, está guardado".

Documentación de la API

Ingresar a la siguiente url <http://localhost:8080/swagger-ui/index.html#/>